

Roteiro de Aula — Laços de Repetição em C

Prof. Bruno Melo · Universidade Positivo — Processo Seletivo Docente

Guia de condução da aula (~20 min) · for, while, do-while

Sumário

Fio condutor único: um simulador de poupança — *quanto tempo até juntar dinheiro para um notebook?* — atravessa todas as etapas (pseudocódigo, slide, quadro e VS Code), dando coesão narrativa à aula.

Como ler este roteiro. Os marcadores em caixa indicam os dois momentos em que você **sai dos slides**: a caixa escura sinaliza **[IR AO QUADRO]** e a caixa azul, **[ABRIR O VS CODE]**. As falas sugeridas estão em caixas cinza. As notas de apresentador também estão embutidas no arquivo de slides (.Rmd): pressione **P** no navegador para o modo apresentador.

1 Abertura / gancho — 00:00 a 02:00 (slides 1 a 4)

Objetivo: criar a necessidade do conceito com uma situação real (MASETTO, 2012).

- **00:00** (capa) — “Bom dia à banca. Sou Bruno Melo. Nos próximos 20 minutos vamos ver como fazer o computador repetir tarefas por nós: os **laços de repetição** em C.”
- **00:20** (roteiro) — Mapa da aula em uma frase: “Começo pela lógica, passo pela sintaxe, vou ao **quadro** e termino com **código rodando ao vivo**.”
- **00:45** (gancho) — Pergunta à banca, com pausa de 2–3 s: “*Quantas vezes você já fez a mesma conta, repetindo à mão?*”
- **01:00** (por que repetir) — Avançar os fragmentos narrando: juntar dinheiro para um notebook; guardar todo mês; conferir o total; “à mão isso cansa”.

Fala-âncora: “A estrutura que expressa *repita* chama-se laço de repetição. É disso que a aula trata: repetir com controle.”

2 Lógica antes da sintaxe — 02:00 a 05:00 (slides 5 a 7)

Objetivo: desenhar o algoritmo em pseudocódigo e plantar a distinção contada x condicionada (ASCENCIO; CAMPOS, 2012).

- **02:00** (transição) — “Primeiro pensamos o problema; depois traduzimos para C.”
- **02:15** (pseudocódigo) — Ler o pseudocódigo apontando cada linha: `total <- 0; ENQUANTO total < meta; acumula; conta meses.` **Frase-chave:** “não sabemos quantos meses serão — isso depende de uma condição. É a repetição **condicionada**.”
- **03:40** (duas perguntas) — Tabela: “Quantos meses até a meta?” leva a **while**; “Quanto em 12 meses fixos?” leva a **for**. Fechar: “mesmo problema, muda só quem decide a parada. Vamos começar pela versão contada, no **for**.”

3 for passo a passo (build interativo) — 05:00 a 09:00 (slides 8 a 14)

Objetivo: iluminar cada parte do laço conforme a narração (inicialização, condição, corpo, incremento). Cada avanço de slide **acende** uma parte do código — aponte a linha destacada.

- **05:15** (visão geral) — “São 3 controles no cabeçalho + o corpo entre chaves.”
- **05:50** (1 — inicialização) — “Acontece **uma única vez**: nasce o contador `mes = 1`.”
- **06:30** (2 — condição) — “**Testada antes** de cada volta. Verdadeiro entra; falso sai. Esse porteiro vira um **losango** no quadro.”
- **07:10** (3 — corpo) — “O **trabalho útil**: acumula a economia e mostra o total. É o `total <- total + mensal` do pseudocódigo.”
- **07:50** (4 — incremento) — “**Ao fim** de cada volta, `mes++`. **Sem isso, a condição nunca muda e o laço fica infinito** (DEITEL; DEITEL, 2011). Guardem essa ideia: é o bug que vou mostrar ao vivo.”
- **08:30** (ordem de execução) — Recitar: inicializa, testa, corpo, incrementa, volta a testar. “É exatamente o que vou desenhar agora.”

4 QUADRO — 09:00 a 12:00 (ponto de pausa planejado)

[**IR AO QUADRO — 3 min**]. Deixe o slide 15 (“AO QUADRO”) projetado: ele sinaliza à banca que a ida ao quadro é intencional. Pegue o giz. O passo a passo completo do que desenhar está no documento **instrucoes_quadro** (PDF). Resumo: desenhe lado a lado os fluxos de **while**, **for** e **do-while**, destacando *onde a condição é testada*.

- **09:00** Desenhar o **while**: início; **losango** `total < meta?`; saída **Verdadeiro** para o retângulo do corpo; **seta de retorno** ao losango; saída **Falso** para “fim”.
- **10:00** Ao lado, o **for**: mesmo losango, marcando onde entram a **inicialização** (antes do teste) e o **incremento** (depois do corpo). “Mesmo fluxo, com contador embutido.”
- **11:00** O **do-while**: colocar o **corpo antes** do losango; a seta de início cai direto no corpo e o losango fica **embaixo**. “Por isso roda **pelo menos uma vez**.”

- **11:40** Circular a diferença: losango **em cima** (while/for) x **embaixo** (do-while).

RETOMADA (11:50): guarde o giz, vire-se para a banca e diga: "Agora que vimos o fluxo no quadro, vamos ver essas três formas escritas em C." **Avance o slide** (15 para 16, bloco **while**).

5 while / do-while — 12:00 a 14:00 (slides 16 a 18)

Objetivo: reforço rápido, amarrando com o desenho do quadro.

- **12:00** (while) — Voltar ao problema da **meta**. “Condição **no topo** — igual ao losango de cima que desenhei. Se o total já bateu a meta, o corpo não roda” (KERNIGHAN; RITCHIE, 1989).
- **12:50** (do-while) — “Aqui o losango está **embaixo**: corpo primeiro, teste depois, roda **pelo menos uma vez**. Pense num menu que precisa aparecer antes de perguntar ‘deseja continuar?’”
- **13:20** (os três lado a lado) — Tabela-resumo: contada x condicionada; quem testa antes/depois; quem pode rodar 0 vez.

6 VS Code ao vivo — 14:00 a 18:00

[**ABRIR O VS CODE — 4 min**]. Alt+Tab para o VS Code, já com as três abas (`exemplo_for.c`, `exemplo_while.c`, `exemplo_erro_proposital.c`) e o **terminal integrado** abertos na pasta `codigos_c`, com a fonte ampliada. O slide 19 ("VS Code ao vivo") fica projetado.

Compilar e rodar (Windows + gcc/MinGW no terminal integrado):

```
gcc exemplo_for.c           -o exemplo_for    ; if ($?) { ./exemplo_for }
gcc exemplo_while.c         -o exemplo_while  ; if ($?) { ./exemplo_while }
gcc exemplo_erro_proposital.c -o exemplo_erro ; if ($?) { ./exemplo_erro }
```

Sem gcc? Use o botão **Run** da extensão C/C++ (Ctrl+F5).

- **14:00 — Exemplo 1 (`exemplo_for.c`):** rodar; digitar 300. “Repetição **contada**: exatamente **12 linhas**, o total crescendo 300, 600, 900.... Fim garantido.”
- **15:00 — Exemplo 2 (`exemplo_while.c`):** rodar; mensal 300, meta 3600. “Repetição **condicionada**: ele para **sozinho** quando bate a meta. Se eu mudar a meta para 5000, ele acha o novo número de meses” (alterar ao vivo se houver tempo).
- **16:00 — Exemplo 3 (`exemplo_erro_proposital.c`) — DEBUG AO VIVO:**
 1. Rodar; mensal 300, meta 3600.
 2. A saída dispara `Mes 1 -> 0.00`, `Mes 2 -> 0.00`, ... **sem parar**. Deixar rolar ~2 s.
 3. **Ctrl + C** para interromper. “Reparem: o total ficou **preso em 0** e o programa **nunca para**.”
 4. Pergunta à banca (avaliação em ação): “*O que está faltando dentro do laço?*” (pausa).

5. Diagnóstico: “A condição testa **total**, mas **nada altera total** no corpo — laço infinito” (DEITEL; DEITEL, 2011).
6. **Correção ao vivo:** descomentar a linha `[FIX] (total += mensal;)`.
7. Recompilar e rodar. “Agora **total** cresce, a condição fica falsa e ele para em 12 meses. **Corrigido.**”

- **17:40** Alt+Tab de volta aos slides.

7 Fechamento — 18:00 a 20:00 (slides 20 a 22)

Objetivo: verificar a aprendizagem e recapitular (MASETTO, 2012).

- **18:00** (verificação) — Ler a pergunta, **silêncio de ~5 s**, convidar a banca a responder. Revelar a resposta: **do-while**, porque testa no fim e executa ao menos uma vez.
- **19:00** (recap) — Percorrer os 5 checkmarks; revelar o encerramento. “Obrigado.”
- **19:40** (Referências) — Deixar à disposição; a bibliografia completa em ABNT está no material de apoio e no plano de aula.
- **20:00** Abrir para perguntas.

8 Planos de contingência

Se estiver atrasado: cortar o slide 17 (do-while em código — o conceito já foi ao quadro); no VS Code, pular o Exemplo 2 e ir do **for** direto ao bug; o slide comparativo C/Python/R é explicitamente complementar.

Checklist pré-aula:

- ☐ `xaringan-themer.css` gerado (knit dos slides uma vez); `up_logo.png` e `fontawesome.html` na mesma pasta.
- ☐ Internet na projeção (ícones) ou aceitar degradação silenciosa.
- ☐ VS Code com 3 abas + terminal na pasta `codigos_c`, fonte ampliada.
- ☐ `gcc --version` respondendo, ou extensão C/C++ configurada.
- ☐ Espaço no quadro para 3 fluxogramas lado a lado; duas cores de giz/marcador.

9 Referências

ASCENCIO, A. F. G.; CAMPOS, E. A. V. **Fundamentos da programação de computadores:** algoritmos, Pascal, C/C++ (padrão ANSI) e Java. 3. ed. São Paulo: Pearson, 2012.

DEITEL, P.; DEITEL, H. **C: como programar.** 6. ed. São Paulo: Pearson Prentice Hall, 2011.

KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação:** padrão ANSI. Rio de Janeiro: Campus, 1989.

MASETTO, M. T. **Competência pedagógica do professor universitário.** 2. ed. São Paulo: Summus, 2012.